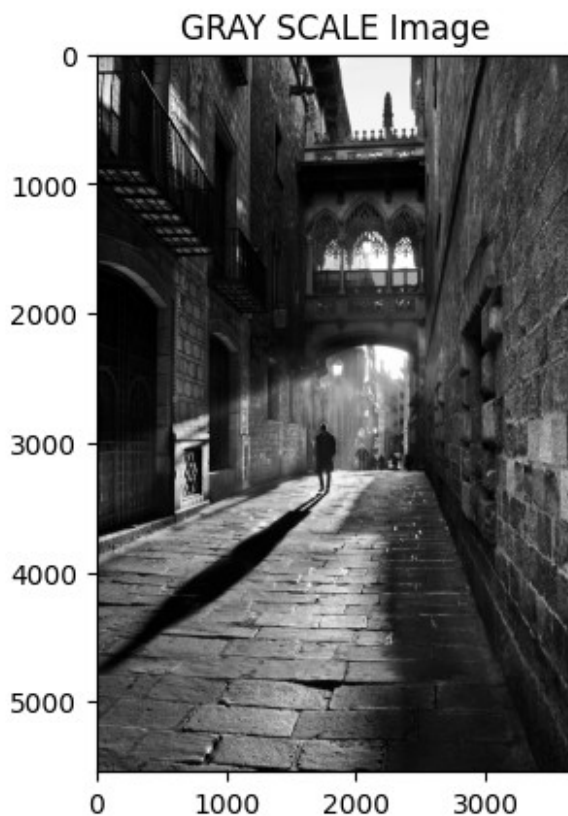


```

!pip install --quiet opencv-python-headless
!pip install --quiet matplotlib
import cv2, numpy as np, matplotlib.pyplot as plt
img = cv2.imread('Gscale.jpg')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
plt.imshow(cv2.cvtColor(img, cv2.COLOR_BGR2RGB));plt.title('GRAY SCALE
Image')

```

```
Text(0.5, 1.0, 'GRAY SCALE Image')
```



```

# Select the intensity range to highlight (e.g., 100 to 200)
min_intensity = 100
max_intensity = 200

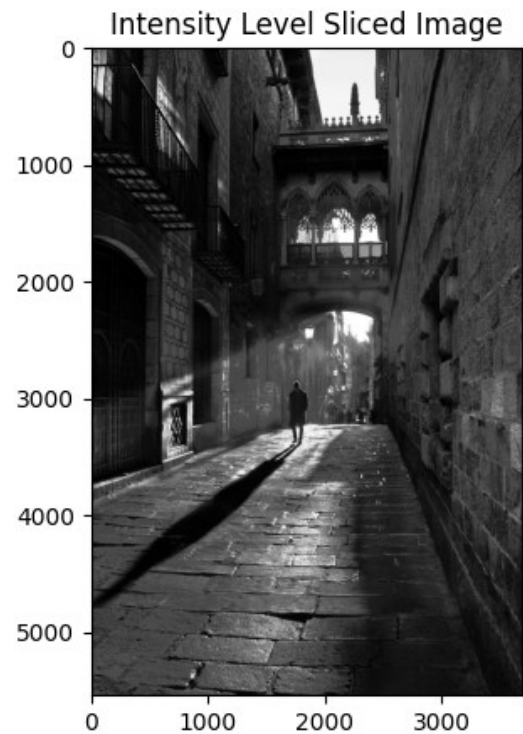
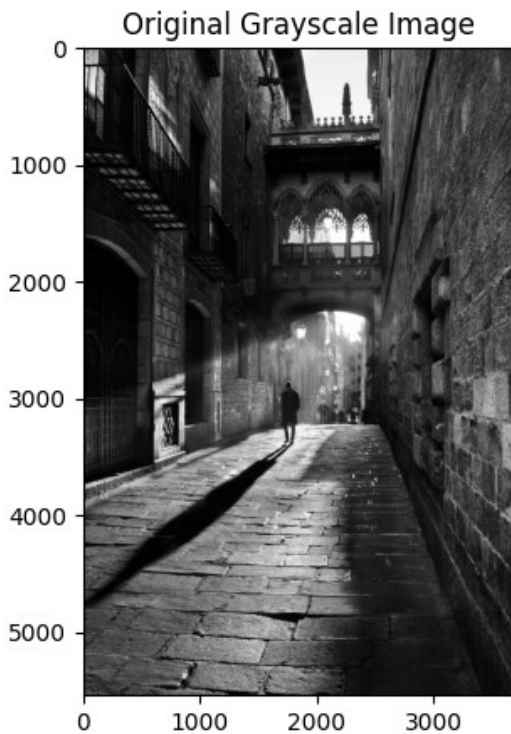
# Create a mask for the selected intensity range
mask = cv2.inRange(gray, min_intensity, max_intensity)

# Increase the brightness of the selected intensity range
intensity_level_sliced_img = gray + mask * 50 # Add 50 to the
intensities within the mask

# Display the original and the intensity level sliced images
plt.figure(figsize=(10, 5))

```

```
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray'); plt.title('Original Grayscale Image')
plt.subplot(1, 2, 2)
plt.imshow(intensity_level_sliced_img, cmap='gray');
plt.title('Intensity Level Sliced Image')
plt.show()
```



```
# Apply histogram equalization
equalized_img = cv2.equalizeHist(gray)

# Compute histograms
hist_original = cv2.calcHist([gray], [0], None, [256], [0, 256])
hist_equalized = cv2.calcHist([equalized_img], [0], None, [256], [0, 256])

# Display the original and equalized images and their histograms
plt.figure(figsize=(12, 6))

plt.subplot(2, 2, 1)
plt.imshow(gray, cmap='gray'); plt.title('Original Grayscale Image')

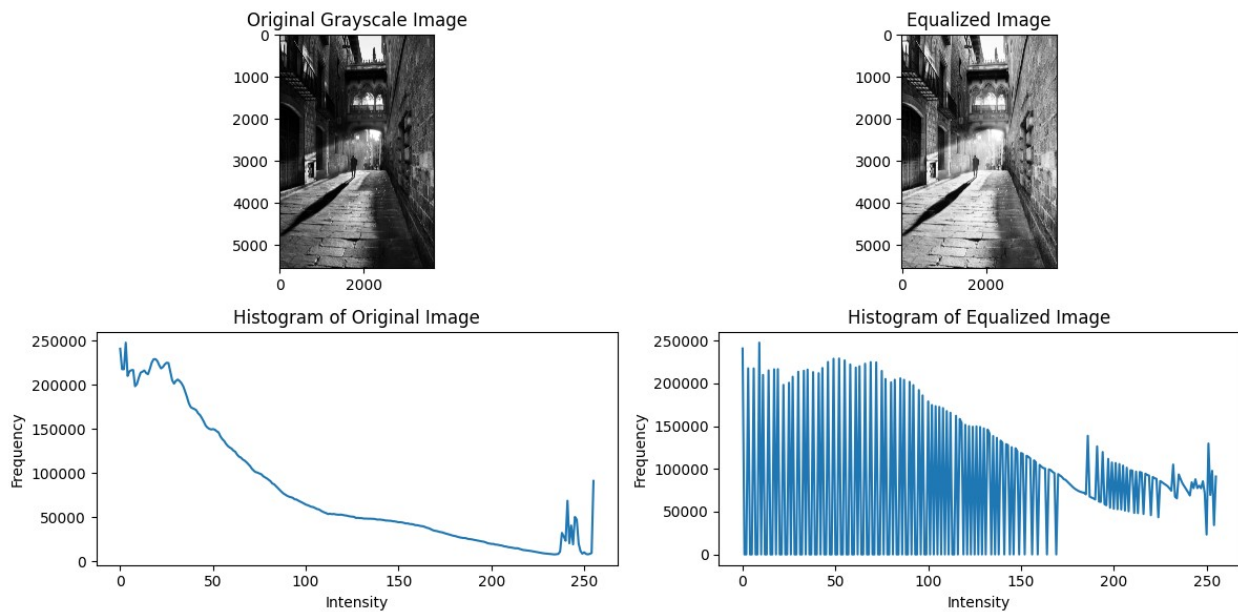
plt.subplot(2, 2, 2)
plt.imshow(equalized_img, cmap='gray'); plt.title('Equalized Image')

plt.subplot(2, 2, 3)
plt.plot(hist_original); plt.title('Histogram of Original Image')
```

```
plt.xlabel('Intensity'); plt.ylabel('Frequency')

plt.subplot(2, 2, 4)
plt.plot(hist_equalized); plt.title('Histogram of Equalized Image')
plt.xlabel('Intensity'); plt.ylabel('Frequency')

plt.tight_layout()
plt.show()
```



```
# Define the mapping points for piecewise linear transformation
# Example: (input_intensity, output_intensity)
mapping_points = [(0, 0), (50, 100), (150, 200), (255, 255)]

# Create a lookup table for the transformation
lookup_table = np.zeros(256, dtype=np.uint8)
for i in range(len(mapping_points) - 1):
    x1, y1 = mapping_points[i]
    x2, y2 = mapping_points[i+1]
    # Calculate the slope and intercept for the linear segment
    m = (y2 - y1) / (x2 - x1) if (x2 - x1) != 0 else 0
    b = y1 - m * x1
    # Apply the linear transformation to the corresponding intensity
    # range
    for j in range(x1, x2 + 1):
        lookup_table[j] = np.clip(m * j + b, 0, 255)

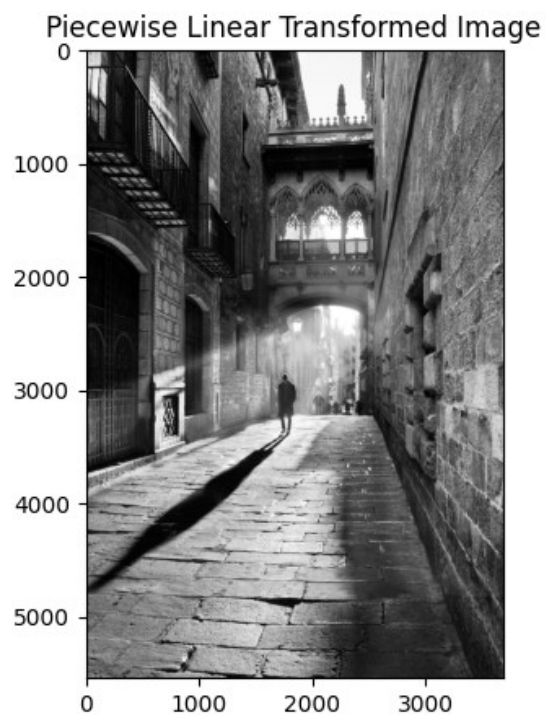
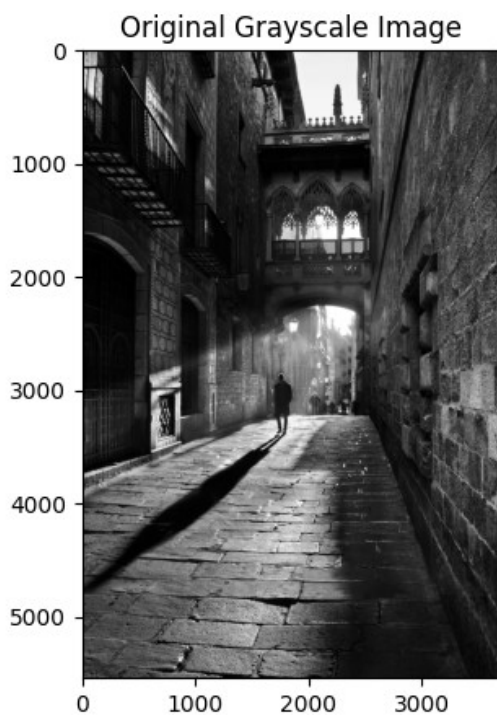
# Apply the transformation to the grayscale image
piecewise_linear_img = cv2.LUT(gray, lookup_table)

# Display the original and the transformed images
```

```
plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.imshow(gray, cmap='gray'); plt.title('Original Grayscale Image')
plt.subplot(1, 2, 2)
plt.imshow(piecewise_linear_img, cmap='gray'); plt.title('Piecewise
Linear Transformed Image')

# Plot the input-output mapping curve
input_intensities = [p[0] for p in mapping_points]
output_intensities = [p[1] for p in mapping_points]

plt.figure(figsize=(6, 6))
plt.plot(input_intensities, output_intensities, marker='o')
plt.xlabel('Input Intensity')
plt.ylabel('Output Intensity')
plt.title('Piecewise Linear Mapping Curve')
plt.grid(True)
plt.show()
```



Piecewise Linear Mapping Curve

